

EXAMEN DE FIN D'ÉTUDES SECONDAIRES CLASSIQUES

Sessions 2023 – QUESTIONNAIRE ÉCRIT

Date :	16.05.23	Durée :	08:15 - 11:15	Numéro candidat :	
Discipline :	Science de la programmation	Section(s) :	CI		

Jeu de Mémoire

Partie modélisation

(10 points)

- La partie modélisation doit être rendue avant que la partie implémentation ne puisse être débütée. Cette partie est à réaliser sur papier.
- La partie modélisation est à rendre après 35 minutes au plus tard, mais vous êtes libre de la rendre plus tôt.

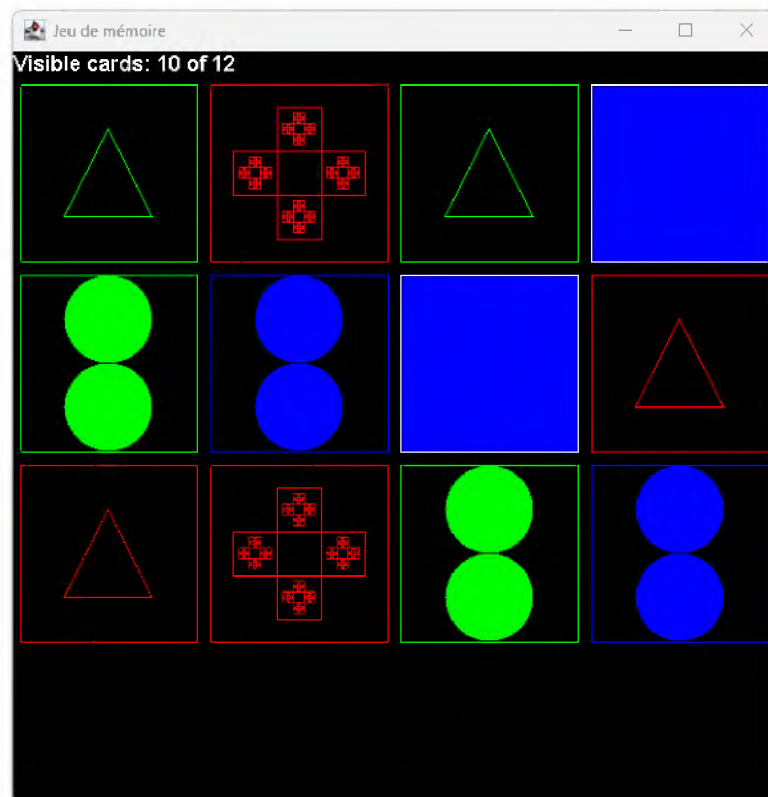
Une société de jeux vous a contacté pour développer un programme permettant de jouer une version du jeu de mémoire (EN : Memory). Le jeu se compose de paires de cartes avec des figures identiques. Les cartes sont mélangées puis placées face cachée. Le joueur retourne deux cartes de son choix. Si les deux cartes ont la même figure, elles restent face visible, mais si l'une des cartes est différente, les deux sont retournées face cachée. Ce processus est répété jusqu'à ce que toutes les faces des cartes soient visibles.

Principe de l'application

L'entreprise met à votre disposition un fichier CSV contenant les différentes cartes. Dans la première version, il y a trois types de cartes, appelées cercles, triangles et carrés. Chaque type dessine une figure différente. Chaque carte a une position, une couleur, une taille et un attribut qui indique si elle est face visible ou non. Pour que deux cartes à jouer puissent être identifiées comme une même paire, il doit être possible de comparer les cartes à jouer entre elles.

La société souhaite une application avec les fonctionnalités suivantes :

1. Au démarrage, l'application lit toutes les cartes à jouer depuis un fichier CSV. Ensuite, toutes les cartes sont mélangées de manière aléatoire.
2. Les cartes à jouer doivent être stockées dans une structure de données qui représente une collection ordonnée dans laquelle un élément donné peut apparaître plusieurs fois.
3. Si un clic de souris se trouve sur une carte de jeu cachée, celle-ci doit être rendue visible.
4. Il doit y avoir un moyen de sauvegarder le jeu actuel dans un fichier CSV.



Travail à réaliser

Réalisez un modèle de classe du modèle de l'application sur papier en respectant les conventions UML usuelles.

1. Ajoutez d'abord les classes nécessaires pour les différentes cartes à jouer avec leurs attributs.
2. Rajoutez dans votre modèle une méthode pour comparer deux cartes à jouer.
3. Ajoutez les méthodes qui sont nécessaires pour sauvegarder et lire toutes les cartes à jouer dans un fichier CSV.
4. Ajoutez les méthodes pour dessiner toutes les cartes à jouer.
5. Ajoutez les méthodes qui rendent une carte visible si une coordonnée d'un clic de souris se trouve sur cette carte.
6. Ajoutez les méthodes pour mélanger toutes les cartes au hasard.
7. Vous pouvez compléter votre modèle de classe par des explications supplémentaires.

Partie implémentation

(50 points)

Dans votre répertoire de travail (à définir par chaque lycée), vous trouverez un dossier nommé EXAMEN_CI. Renommez ce dossier en remplaçant le nom actuel par votre code de l'examen (exemple de notation : LXY_CI2_05). Tous vos fichiers devront être sauvegardés à l'intérieur de ce dossier, qui sera appelé « votre dossier » par la suite.

Vous allez programmer dans la suite une application permettant de simuler une version du jeu de mémoire. Pour simplifier un peu le jeu lorsque l'on clique sur la deuxième carte, celle-ci n'est visible que si elle est identique à la première, sinon les deux cartes sont tout de suite cachées. Ouvrez dans votre dossier le projet **Memory** mis à disposition dans votre dossier. Dans le dossier du projet, vous trouvez également un fichier CSV appelé « memory.csv », qui peut être utilisé comme exemple pour tester votre implémentation. Complétez l'application en vous basant sur la version exécutable, le diagramme UML et les instructions données par la suite.

La classe « Card »

8 points

La classe abstraite **Card** représente une carte carrée du jeu de mémoire. Cette carte peut être soit face visible et donc montrer une figure, soit face cachée et donc ne pas montrer une figure. La classe possède les attributs suivants :

- **visible** indique si la figure est visible (face visible) ou si elle est cachée (face cachée). La valeur initiale est fausse.
- **color** représente la couleur de la figure.
- **position** représente le coin supérieur gauche de la carte.
- **size** correspond à la longueur des côtés de la carte.

Le **constructeur** initialise la valeur des attributs **color**, **position** et **size**.

0.25p

La classe possède les méthodes suivantes :

- Ajoutez les accesseurs et les manipulateurs comme décrit dans le schéma UML. 0.25p
- La méthode **toCsv** retourne la représentation textuelle de la carte au format CSV, qui correspond au format suivant : « Nom de la classe » ; « valeur rouge de la couleur » ; « valeur verte de la couleur » ; « valeur bleue de la couleur » ; « coordonnée x de la position » ; « coordonnée y de la position » ; « taille ». 2p

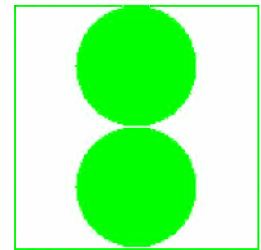
Remarque : L'utilisation du code **getClass().getSimpleName()** permet de récupérer le nom de la classe dont l'objet a été instancié.

- La méthode **isInside** indique si la coordonnée indiquée en paramètre se trouve à l'intérieur de la carte. 2p
- La méthode **draw** est responsable du dessin de la carte. Si la carte n'est pas visible, un carré bleu de la taille indiquée dans l'attribut **size** est dessiné avec un bord blanc, sinon la figure elle-même est dessinée et le bord de la carte prend la couleur de la figure. Utilisez l'héritage le plus possible. 1.5p
- La méthode **areSamePair** permet de vérifier si deux cartes sont de la même paire à l'aide du nom de classe et de la couleur. Si le nom de classe et la couleur des deux objets sont identiques, alors les deux cartes sont de la même paire. Si ce n'est pas le cas, les deux cartes sont différentes. 2p

La classe « Circles »

3 points

Cette classe est une classe fille de la classe **Card** et dessine deux cercles superposés comme figure mais uniquement si la carte est visible. La classe possède l'attribut **radius** qui est initialisé dans le constructeur avec un quart de la taille de la carte. Ajoutez le constructeur comme décrit dans le schéma UML.



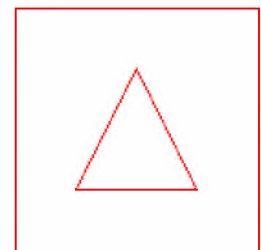
- La méthode **draw** dessine les deux cercles superposés avec la couleur indiquée dans l'attribut **color**. L'intersection des deux cercles (milieu de la figure) se trouve au milieu de la carte. Le rayon des deux cercles est identique et est précisé dans l'attribut **radius**.

3p

La classe « Triangle »

4 points

Cette classe est une classe fille de la classe **Card** et dessine un triangle isocèle comme figure si la carte est visible. Ajoutez le constructeur comme décrit dans le schéma UML.



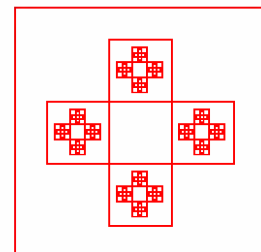
- La méthode **draw** dessine un triangle isocèle avec la couleur indiquée dans l'attribut **color**. La base et la hauteur du triangle ont toutes les deux la même taille, qui est la moitié de la taille de la carte. Le sommet du triangle est orienté vers le haut et le triangle est situé au centre de la carte.

4p

La classe « Squares »

5.5 points

Cette classe est une classe fille de la classe **Card** et dessine des carrés imbriqués comme figure si la carte est visible. Ajoutez le constructeur comme décrit dans le schéma UML.



- La méthode **draw** dessine un carré, puis dessine quatre nouveaux carrés à l'intérieur du carré précédent. Si la taille du rectangle est inférieure à 2, le processus s'arrête. Tous les carrés ont la même couleur, qui est indiquée dans l'attribut **color**. La taille de chaque nouveau carré correspond à un quart de la taille du carré précédent. Le centre de chaque nouveau carré est situé à un quart de la taille du carré parent, horizontalement ou verticalement par rapport à son centre. Cette méthode doit être implémentée de manière récursive et il est permis d'ajouter d'autres méthodes à la classe **Squares**.

5.5p

La classe « Node »

1 point

La classe **Node** représente un nœud dans une liste chaînée dans laquelle une carte du jeu peut être enregistrée. Implémentez la classe **Node** selon le schéma UML. La classe contient un constructeur, des accesseurs et des manipulateurs.

La classe « Memory »

25 points

La classe **Memory** contient toutes les cartes du jeu et permet de les lire à partir d'un fichier CSV et de les enregistrer dans un fichier au format CSV. La classe possède les attributs suivants :

- **root** représente le premier nœud de la liste.
- **size** correspond à la taille de la liste.
- **currentSelectedCard** indique la carte actuellement sélectionnée.

La classe possède les méthodes suivantes :

- La méthode **add** ajoute une nouvelle carte au début de la liste. Ensuite, **size** est incrémenté d'une unité. 1.5p
- La méthode **get** retourne le nœud qui se trouve à la position indiquée par le paramètre **index**. Si l'index n'est pas valide, une exception de type **IndexOutOfBoundsException** est lancée avec un texte significatif. 3p
- La méthode **getRandom** génère un nombre aléatoire compris entre min et max inclus. 0.5p
- La méthode **shuffle** mélange les cartes de manière aléatoire. Le paramètre **n** indique le nombre de fois où deux cartes doivent être permutées entre elles. La permutation s'effectue en échangeant les valeurs des attributs « **position** » entre les cartes. Veillez à réduire à un minimum le nombre d'opérations qu'effectue cette méthode ! 3p
- La méthode **saveToFile** écrit toutes les cartes du jeu de la liste dans un fichier CSV. Le nom du fichier est indiqué par le paramètre **filename**. 2P
- La méthode **loadFromFile** lit toutes les cartes du jeu à partir du fichier CSV. Le nom du fichier est indiqué par le paramètre **filename**. Toutes les exceptions possibles sont transmises à la classe appelante. Le format d'une ligne est le même que celui généré par la méthode **saveToFile** (voir **Card.toCsv()**). 4p
- La méthode **getNumberOfVisibleCards** compte toutes les cartes de la liste pour lesquelles la figure est visible (face visible). 2p
- La méthode **drawAll** dessine toutes les cartes et écrit en blanc à la position (0, 15) le nombre de cartes visibles avec le nombre total de cartes. Voir le programme de démonstration dans votre dossier. 2p
- La méthode **verifyPair** vérifie si la carte passée en paramètre est de la même paire que la carte actuellement sélectionnée. Tout d'abord, la carte passée en paramètre est rendue visible. Ensuite, il y a les cas suivants :
 - Si aucune carte n'a encore été sélectionnée, la carte passée en paramètre devient la carte actuellement sélectionnée.
 - Si la carte passée en paramètre et la carte actuellement sélectionnée sont exactement la même (équivalent), la méthode se termine.
 - Si les deux cartes n'appartiennent pas à la même paire, elles sont à nouveau cachées et aucune carte est sélectionnée.
 - Si les deux cartes appartiennent à la même paire, elles restent face visible et aucune carte n'est sélectionnée. 5p
- La méthode **click** recherche la carte dont le point indiqué en paramètre se trouve à l'intérieur. Si une telle carte existe et si elle est encore cachée, la méthode vérifie si cette carte fait partie d'une paire. 2p

La classe « DrawPanel »**1 point**

La classe **DrawPanel** est mise à votre disposition et doit être complétée. La classe dessine le jeu de mémoire. Elle possède l'attribut du type **Memory** et le manipulateur correspondant. Le jeu est dessiné sur un fond noir.

La classe « MainFrame »**2.5 points**

La classe **MainFrame** elle-même est mise à votre disposition et contient déjà le code que vous devez compléter selon les instructions suivantes. Les exceptions possibles sont traitées en affichant le message dans un dialogue (`JOptionPane.showMessageDialog(this, MESSAGE);`).

- La classe possède l'attribut **memory**, qui représente le jeu de mémoire.
- Au démarrage du programme, toutes les cartes doivent être chargées à partir du fichier « **memory.csv** » et ensuite mélangées 100 fois. 1p
- Si on relâche un bouton de la souris sur le **drawPanel**, la méthode **click** de la classe **Memory** est appelée avec les coordonnées de la souris. Complétez la méthode. 0.5p
- En relâchant la touche 's', le jeu actuel est enregistré dans le fichier « **memory.csv** ». Une partie de la méthode est déjà implémentée. Complétez la méthode à l'endroit indiqué dans le code. 1p

